

Touchless SDK
&
Multi-Touch Vista

Markowitz Frost
Advisor: Dr. Yu-Ju Lin
April 20, 2012

Table of Contents

Introduction.....	2
Purpose of selecting Project.....	2
Summary of Touchless	2
Summary of Multi-Touch Vista.....	3
Project Timeline.....	4
Research.....	5
Touchless SDK	5
Multi-Touch Vista	6
Touchless SDK	
Background.....	6
Issues and Solutions.....	6
Advantages and Disadvantages	7
Multi-Touch Vista	
Background.....	8
Issues and Solutions.....	8
Advantages and Disadvantages	8
Improvements	
Pointer Algorithms for mouse	9
Search Algorithms for Touchless SDK	10
Interface Design.....	10
Future Improvements.....	11
References.....	12
Appendix A	
How to use Touchless Implementation	
How to play Snake Game	
Appendix B	
Implementation Code	
Snake Game Demo Code	
Appendix C	
Touchless SDK Documentation	
Touchless SDK Class Diagram	
Multi-Touch Vista Layer Architecture Diagram	

INTRODUCTION

A significant portion of the technology world is aware of the Microsoft Kinect, designed for Xbox 360, and what features it offers to developers and users. These same people are also aware of the effort Microsoft has put towards advertising the possibility of what the Kinect can do and what it has to offer the world. They recently released a Kinect SDK for Windows to allow users to explore the possibilities of the hardware which included integrating it with Windows 7/8. The disadvantage behind this is that a developer would have to purchase an expensive piece of hardware to gain this functionality. The purpose behind this project is to allow users or developers to use the hardware already included as standard in most computers to perform similar functions. You would be able to use your webcam to control Windows 7/8 in the same manner as you would if you were using a touch screen, but the cost for the extra hardware would not be necessary.

PURPOSE OF SELECTING PROJECT

This project was initially started by Michael Lord who made significant progress and was able to get the project off the ground in stable form. However, I was not completely satisfied with the end result because the project looked as if it had much more potential and had many other implementations that it could be used for. The project caught my interest because I knew I could optimize it to have smoother mouse movements and to be less CPU intensive. The algorithms that were being used were good, but there were many other tricks that I learned in my other programming classes that could be used to ensure that the result would be a smooth and easily useable mouse for anyone.

Throughout my research for this project, I learned about the Multi-Touch Vista project that could be implemented with Touchless to allow use of Windows' Multi-Touch and gesture functions. This piqued my curiosity and gave me the idea to also try and implement this into my project because in the end that seemed to be what Touchless was geared towards. From there, I completed the optimization of the Touchless SDK project and began implementing the Multi-Touch Vista into my project.

BACKGROUND OF TOUCHLESS SDK

The rationale behind why the Touchless SDK project was initially developed was because it was supposed to allow users to create and experience multi-touch applications. It was to offer users a cheap and new way to experience the multi-touch capabilities without the need of expensive hardware or software. The project was started by Mike Wasserman as a college project at Columbia University. The project is now released for free and open-source to the world under the Microsoft Public License to drive community involvement and continue to develop the usefulness of the project.

BACKGROUND OF MULTI-TOUCH VISTA

Multi-Touch Vista is a user input management layer that handles input from various devices and libraries and normalizes it against the scale and rotation of the target window. It has now been updated to use the multi-touch driver for Windows 7.

PROJECT TIMELINE

▲ May 2011 – June 2011

- ☐ Research image processing
- ☐ Learn how to program in C#

▲ August 2011 – October 2011

- ☐ Reading over code and notes from Michael Lord to mark places to improve upon

▲ November 2011 – December 2011

- ☐ Revise image searching algorithm and begin planning for additional features

▲ January 2012 – April 13, 2012

- ☐ Work on making process efficient and less CPU intensive
- ☐ Allow customization of settings of the program
- ☐ Smooth out data from position detection
- ☐ Allow Multitouch capability.
- ☐ Allow Win7 Multi-touch gestures to be usable

▲ February 2012

- ☐ Write documentation for Touchless SDK portion
- ☐ Begin Implementing Multitouch ability using MultiTouch Vista SDK

▲ March 2012

- ☐ Work out Touchless and Multi-touch Vista bugs
- ☐ Testing of full functionality within Win7

▲ April 2012

- ☐ Prepare presentation and complete documentation.

RESEARCH

A portion of the research was already completed by Michael Lord, who previously worked on this project; it contained most of what I needed to completely understand how the Touchless environment worked. Due to the complexity of what I wanted to implement, I had to do further research into figuring out what would be necessary to smooth out the data that the program sent to the mouse. This included figuring out what would be necessary to cut down on the resources it used to process the image and how to alter the algorithm it used to search for markers. Alongside the algorithm research, I also needed to make improvements to how the cursor moved, so that it could move smoothly so that it could be useful by users.

The next major topic to research for this project was how to implement a way to allow the Touchless SDK to talk to Windows to allow it to use the multi-touch functions. This led me down the road to discover the Multi-Touch Vista project, which was somewhat early in its development stages. After discovering this, I then proceeded to figure out how to get the Touchless SDK to talk to Multi-Touch Vista to enable the multi-touch functionality in the operating system.

TOUCHLESS SDK

The main drawback for the Touchless SDK is that its algorithms are limited to using only what Windows offers it. This meant that the current way that Touchless saved images and searched through them would remain the same, copying them on to the Windows clipboard and scanning them from there. Since I soon learned that there was no easy way to modify the library to use its own space in memory to store and scan the images, I had to find another way to cut back on resources. This led me to the idea of splitting the image into 4 parts and having individual threads scanning the image to increase the search speed and bring down the CPU usage.

In search to find out how other similar programs or systems performed this task, the Microsoft Kinect came to mind to better understand how it performed this task. In a Microsoft Research article they mention the sheep-and-grass example where, "You take a photo of a sheep on grass and try to segment the sheep against the grass and label those automatically as sheep and grass." (Knies). This along with using the Kinect's depth sensor helped figure out the shape of the object it was looking at and designated it a certain color. This methodology would be effective in the Touchless SDK, but it would take far too long to modify the library to implement such an algorithm. It would have the potential to speed up the search through the images to discover where the marker is located.

Another technology more akin to what the Touchless SDK is doing is the Playstation Move. The way that the Move system works is by using the Playstation EyeToy (webcam) to track a glowing ball of a specific color at the end of a controller and uses accelerometer and gyroscope data to determine its orientation and position. Sony has not officially released the source for the method they use for tracking. Their method for image search could be implemented into the Touchless environment, but the downfall to their method is that it works by searching for those glowing balls based on their specific color. Since Touchless will be able to

look for any color the user defines, that implementation would need to be adjusted to suit those needs.

MULTI-TOUCH VISTA

Multi-Touch Vista is better described as an input manager rather than an SDK due to the functionality of the program. It takes input from custom devices or programs and forwards that input to the Windows Presentation Foundation (WPF) to be translated into touch inputs recognized by Windows. Building a program to work directly with Multi-Touch Vista involved creating a “WPF Application” in Visual Studio, this was necessary to effectively use the Multi-Touch Vista project inside an application rather than in the full operating system environment.

There are few programs that are similar out there that allow this type of integration into the operating system, of the ones out there, many are catered to a specific input device. The moveonpc project is designed to allow you to use the Playstation EyeToy to allow you to track the position, angle, and depth of one of the Playstation Move controllers. It is a framework in development waiting on the community to expand on its capabilities which could most likely include integrating it into the Windows operating system environment.

TOUCHLESS SDK

BACKGROUND

My purpose behind working on Touchless SDK was to improve its usability so that it could be practical and so that users wouldn't feel that the control scheme was too cumbersome to use.

ISSUES AND SOLUTIONS

1. The raw data from the scanned image was not accurate.
Essential to solving this issue was to discover what options the Touchless SDK offered for what it does with the rendered data involves. The options that were included that initially effected the results of the data were setting Smoothing and Threshold. Setting these two values based on the object or color that was selected helped resolve the problem of the mouse recognizing any mildly similar colors in the background. To further increase the accuracy of the marker detection, the radius of the marker was no longer a static 5 pixels in size. I altered the interface so that when you wanted to set a marker, you would click and drag a circle to give the algorithm a more accurate radius to use to determine the accuracy of the object it is tracking.
2. The relative mouse position algorithm was not correct.
The purpose of the relative algorithm was to make the Touchless project respond similar to a trackpad. This was so that if you lifted your finger and placed it somewhere else on the trackpad and moved it, the position of the mouse would change based on the change in position of your finger. So in the case of Touchless, if the marker was not seen by the

camera, then it came back into view, it would not jump to the same relative position based on what the camera sees, it would move only when the marker changed. To accomplish this, it was necessary to use the data set from the previous frame to determine the change. The mouse would then move a set number of pixels based on the change from the previous frame and from the sensitivity variable that I put in place. With these changes for the relative mouse, it provided a good simulation to how a trackpad would behave.

3. Mouse movement was very sporadic and not smooth.

Resolution to this issue came in the form of implementing separate threads for the mouse movements that would move the mouse in 10 steps, or change the position of the mouse 10 times before the thread ended. To ensure the movement was truly smooth, the threads that were running were put to sleep for 5ms to simulate a dragging of the mouse, otherwise the threads would run so fast that the mouse would jump to its position each time. Within the new threads, I thought I would have had to use some advanced math to get accurate and natural movement from the cursor. It turned out that all I needed to do was have the cursor to change based on a linear equation because the changes are so small and so quick that the user would not notice the difference.

4. Adding a second marker dramatically slowed down processing

This issue has not been solved yet, the cause of it resides in the library reading the image and scanning it once for each marker which puts a major strain on I/O when the webcam is capturing at 30 frames per second.

ADVANTAGES AND DISADVANTAGES

Since Microsoft's XNA game studio's primary coding language is C#, if someone would want to include this in their implementation of the game. It could be much easier to implement directly into the game rather than having to pass through Direct X to talk to the game just for it to talk back to Direct X. The accessibility into the system that is available to C# would make it simpler to implement thoroughly into games for usage as simple as a joystick or as complicated as navigating a 3D menu system. There would not be much overhead since Touchless only stores the current image from the webcam and the previous values generated from the raw data. Ease of use is the greatest advantage to using Touchless, it's easy to jump into and get a quick project started, but also provides the depth to allow you to completely customize how you want it to perform.

On the other side of things, Touchless is limited to how deep into the framework you will be able to modify how the algorithms work. The library is not locked down by any means, but much of the functionality is intertwined together. Because this is the case, it would be very complicated to implement this SDK for usage such as Face tracking or tracking objects of unique shape. Another result of it only tracking colors is that it may also accidentally track another object in the room of a similar color. On the performance side, you would need to run this on a computer that has at least a 1.5 GHz dual core processor with fast memory. This issue is countered by the Kinect by the fact that it has its own dedicated processor to process each frame, which is not the case for Touchless.

MULTI-TOUCH VISTA

BACKGROUND

Having the option to use any input device you want is the goal of the Multi-Touch Vista program; it acts as the mediator between your input device and the Windows multi-touch framework.

ISSUES AND SOLUTIONS

Multi-Touch Vista was built with a very robust system; there were very few occasions where I believed there were issues with Multi-Touch Vista. Many issues or errors that came up were because of the proper command not being sent to Multi-Touch Vista to tell the WPF layer so that the touch features could be used. The largest issue with using it is if you want to use your own custom interface then you will have to rely on your own abilities to determine how to properly have your input device or program talk to it.

Another issue that may come up is if you want to develop an application that interacts directly with Multi-Touch Vista, it would be necessary to learn how to code in XAML. This is the language that WPF uses to design its forms to be interactive with a multi-touch interface. It is similar in style to using HTML because it is a markup language, but it offers much deeper functionality than HTML.

ADVANTAGES AND DISADVANTAGES

The clear advantage that Multi-Touch has is that any commands that are sent to it need to be in some uniform way so that it can talk to the WPF. In most cases this would make it easier to develop a multi-touch device to give you full access to all of the functions that Windows has to offer. Like a traditional touch screen device, it can eliminate the need for use of a mouse and possibly a keyboard as well. Because the program relies on standardized commands from some very common input providers, when designing a new input device there is a lot of support available to ensure your new device adheres to the standards already set in place.

Since Multi-Touch Vista uses the Windows Presentation Foundation, there is no way to port this technology over to other operating systems like Macintosh OS X or any Linux distributions. That is not a deal breaker for deciding if it would be a good program to use to develop a new input device with though. But if you do want to develop your own custom input device, there is not much support for those wanting to send custom commands to it to perform unique operations outside of what's given.

IMPROVEMENTS

POINTER ALGORITHMS FOR MOUSE

For a user to consider an interface good and usable, it must first be responsive, and then it must also be smooth. In this case the original algorithm for moving the mouse caused the mouse to jump around a lot to its position once the camera found it. This was solved by placing the function of moving the mouse across the screen in a separate thread from the main program. The other issue in the mouse algorithm was getting the relative mouse movement functionality to work properly, this was done by using the previous data that was used by Touchless. Since the only difference between the algorithms was where they would be placing the cursor, they were placed in separate 'if' statements and have the cursor move afterwards.

```
PointF iterPoint = Cursor.Position;
    PointF newpos;
    if (!relativity) //absolute pointer
    {
        //Calculates X,Y in relation to the scaled sizes between main window and
the form window
        newpos = new
PointF((float)(System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width - (e.EventData.X *
(System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / pictureBox1.Width))*1.2),
        (float)(e.EventData.Y *
(System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / (pictureBox1.Height - 60))));
    }
    else //relative pointer
    {
        //Uses the Change in X,Y (DX,DY) to determine movement distance
        newpos = new PointF((float)(Cursor.Position.X - (e.EventData.DX *
(System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / pictureBox1.Width))),
        (float)(Cursor.Position.Y + (e.EventData.DY *
(System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / pictureBox1.Height))));
    }

    //Difference of new point and current point to determine the direction
and distance to move
    PointF slope = new PointF(newpos.X - Cursor.Position.X, newpos.Y -
Cursor.Position.Y);

    new Thread(() =>
    {
        slope.X = (slope.X * SENSISTIVITY) / steps; //X & Y coordinates that
are added each step
        slope.Y = (slope.Y * SENSISTIVITY) / steps; //to move the cursor in a
linear direction
        for (int i = 0; i < steps; i++)
        {
            //New point for cursor to jump to
            iterPoint = new PointF(iterPoint.X + slope.X, iterPoint.Y +
slope.Y);
            Cursor.Position = (Point.Round(iterPoint)); //Send cursor X & Y
of next point
            Thread.Sleep(5);
        }
    }).Start();
```

SEARCH ALGORITHMS FOR TOUCHLESS SDK

Searching through an image in Touchless meant scanning the entire image from each frame captured from the camera to find the new marker. This was clearly inefficient and time consuming, but the good thing about how the SDK was set up is that the search bounds were defined in a separate function to determine how large of a region that the search function should look through. The solution to this problem meant limiting the search bounds as much as possible so that it would only have to search a small area of the image. To do this, the information from the data collected from the previous frame would be used, in particular the changes in X and Y would be used to help determine how large the search bounds should be.

```

if (marker.prevData.Present)
{
    // TODO: Improve the search bounds, consider acceleration...
    int xBuffer = imageWidth / 20;
    int yBuffer = imageHeight / 20;

    float dxFactor, dyFactor;
    if (marker.prevData.DX < 0) { dxFactor = 0.25F; }
    else { dxFactor = 4; }
    if (marker.prevData.DY < 0) { dyFactor = 0.25F; }
    else { dyFactor = 4; }
    marker.searchMinX.coordinate = (int)(marker.prevData.Left +
marker.prevData.DX / dxFactor - marker.prevData.Width / 3 - xBuffer);
    marker.searchMaxX.coordinate = (int)(marker.prevData.Right +
marker.prevData.DX * dxFactor + marker.prevData.Width / 3 + xBuffer);
    marker.searchMinY.coordinate = (int)(marker.prevData.Top +
marker.prevData.DY / dyFactor - marker.prevData.Height / 3 - yBuffer);
    marker.searchMaxY.coordinate = (int)(marker.prevData.Bottom +
marker.prevData.DY * dyFactor + marker.prevData.Height / 3 + yBuffer);

    // Ensure the marker search bounds are within the image region
    marker.searchMinX.coordinate = Math.Max(marker.searchMinX.coordinate, 0);
    marker.searchMaxX.coordinate = Math.Min(marker.searchMaxX.coordinate,
imageWidth - 1);
    marker.searchMinY.coordinate = Math.Max(marker.searchMinY.coordinate, 0);
    marker.searchMaxY.coordinate = Math.Min(marker.searchMaxY.coordinate,
imageHeight - 1);
}
else
{
    marker.searchMinX.coordinate = marker.searchMinY.coordinate = 0;
    marker.searchMaxX.coordinate = imageWidth - 1;
    marker.searchMaxY.coordinate = imageHeight - 1;
}

```

INTERFACE DESIGN

The interface was given more options so that the user could have more control over how the mouse interacted with Touchless, these options that were added are similar to what one would find on the settings for a touchpad device. The inclusion of sensitivity only affects the

relative pointer because including it with the absolute algorithm only served to cause a lot of jumping when the mouse moved.

FUTURE IMPROVEMENTS

TOUCHLESS SDK

The Touchless SDK is very flexible with its implementation, as shown already, it can be used to control functions of the user interface or it could even be used as a secondary controller for games. It could be implemented in a similar fashion like the Playstation Move, since it is also tracking the color of an object, except it has the additional data of the orientation of the controller from the gyroscope and accelerometer. Along with Windows 8 on the horizon, the touch-centric interface of it would be perfect for what Touchless has to offer. This would give the user the ability to interact with Windows on their laptop or desktop in the same way they would if they were using a touch screen computer or phone.

There could be more improvements to my implementation of this project, someone could possibly figure out a way to make it more efficient and setting it up to have its own gesture algorithms so that it can do its own gestures. With that available to the program, it would eliminate having to use Multi-Touch Vista and being limited to the gestures Windows 7 has to offer.

MULTI-TOUCH VISTA

Multi-Touch Vista at its core is an interface management program designed to allow your custom device to interact with the Windows 7 multi-touch layer. With this in mind, there is not much improvement that would need to be done on any of the algorithms within the offered development tools for it. However, ability of being able to use all the mice you need opens it up to possibilities of being able to develop a device similar to the Microsoft Surface so you can use a large screen on a table as a large tablet.

RESOURCES

Knies, Rob. "Kinect Body Tracking Reaps Renown." Microsoft Research, 6 Sept. 2011. Web. <<http://research.microsoft.com/en-us/news/features/kinectskeletal-092711.aspx>>.

Anton Mikhailov. (2009-08-31) (Flash Video). PlayStation Motion Controller Interview Part 2. [podcast]. Foster City, California: Sony Computer Entertainment America. <<http://www.viddler.com/explore/sceablog/videos/546/>>.

hkaufmann@gmail.com. "Supporting Playstation Move on PC." MoveOnPC. 19 Sept. 2010. Web. <<http://code.google.com/p/moveonpc/>>

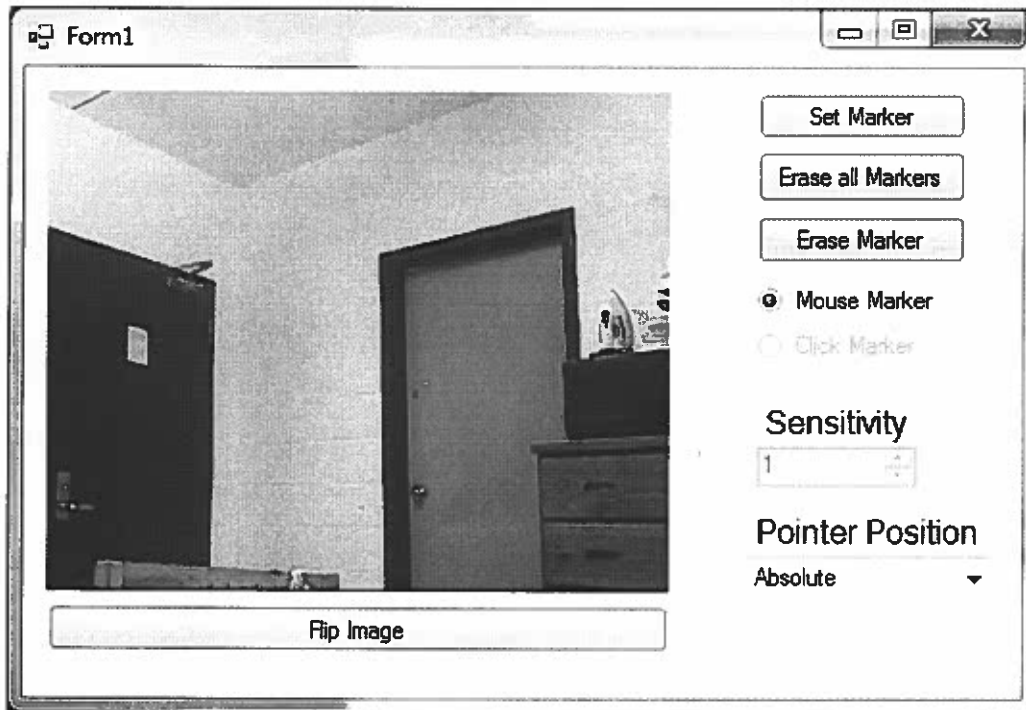
Nesher. "Multi-Touch Vista." 12 April. 2008. Web. <<http://multitouchvista.codeplex.com>>

Wasserman, Mike. Touchless SDK. 23 Sept. 2008. Web. <<http://touchless.codeplex.com>>

How to use Touchless Implementation

Mark Frost

1. When the program starts up initially, you are presented with a screen showing the video from your webcam along with a few options to choose from on the side of the video. The screen should look like this initially:



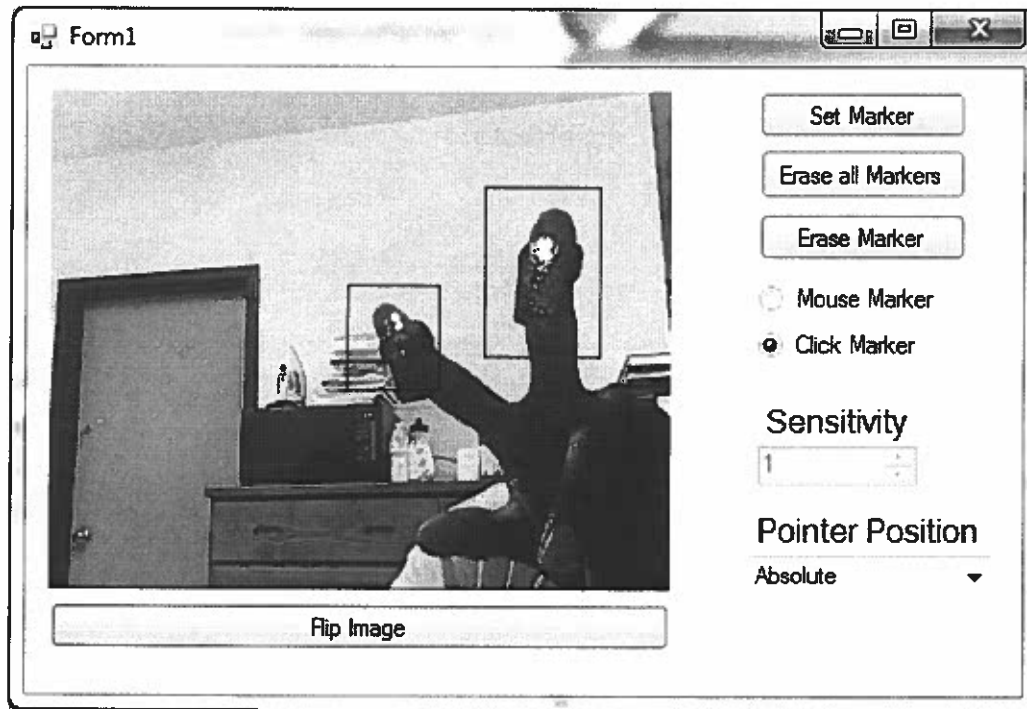
Note: The sensitivity option is grayed out unless you choose for the pointer position to be relative. And the 'Click Marker' option will be grayed out until the 'Mouse Marker' has been set.

2. For the next step, you are going to want to place the object you intend to use as your marker in front of your webcam so we can set it. Once it is in front of your camera press the 'Set Marker' button, this will cause the image from the webcam to freeze allowing you to set your marker. To set the marker accurately, use the mouse to click and drag from the center of your marker until the radius of the circle encompasses a majority of the color of the object, and then release the mouse button.



Note: For improved accuracy tracking the marker, you will want to be in a well lit room and you will want your marker to be a color contrasting any colors in the background. Be sure that the size of the circle is not larger than it needs to be.

3. Once set, the 'Click Marker' option becomes available and you will notice a box tracking your marker. Setting the Click Marker is the same process as setting the Mouse Marker, refer to the first step to complete this process.



4. After the Click Marker is added, and they are both held up in front of the screen you will see the two rectangles showing that the markers are being tracked. If the result of this process is what is shown then you have successfully setup the Touchless interface.
5. You now have the option to change the pointer position to Relative from the drop-down box, when this is set, the Touchless interface will act in a similar manner to a touchpad. The equivalent to lifting your finger off a touchpad is making the marker disappear from the webcam's view.

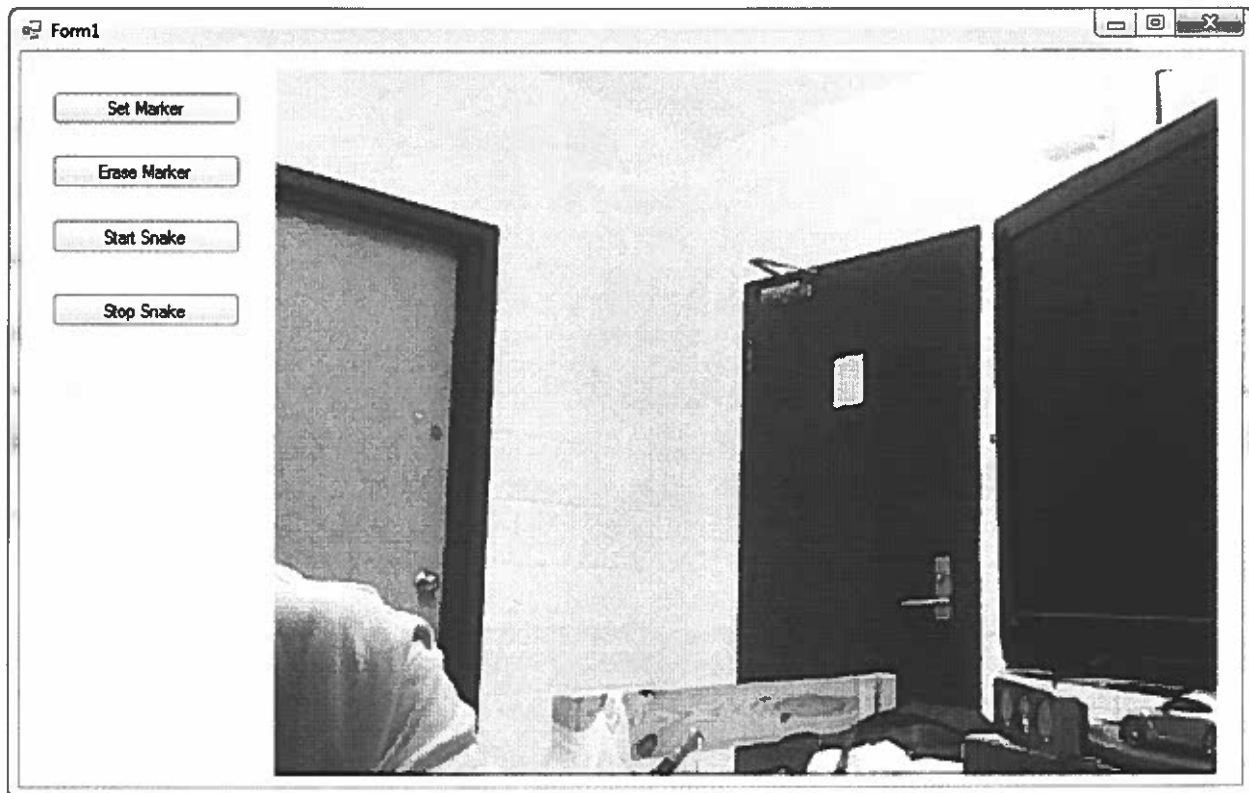
When the pointer position is in Relative mode, then you will be able to set how sensitive you want the mouse movement to be. The range of 1 to 10 (lowest to highest) is the available settings for the sensitivity.

Now go out and have fun!

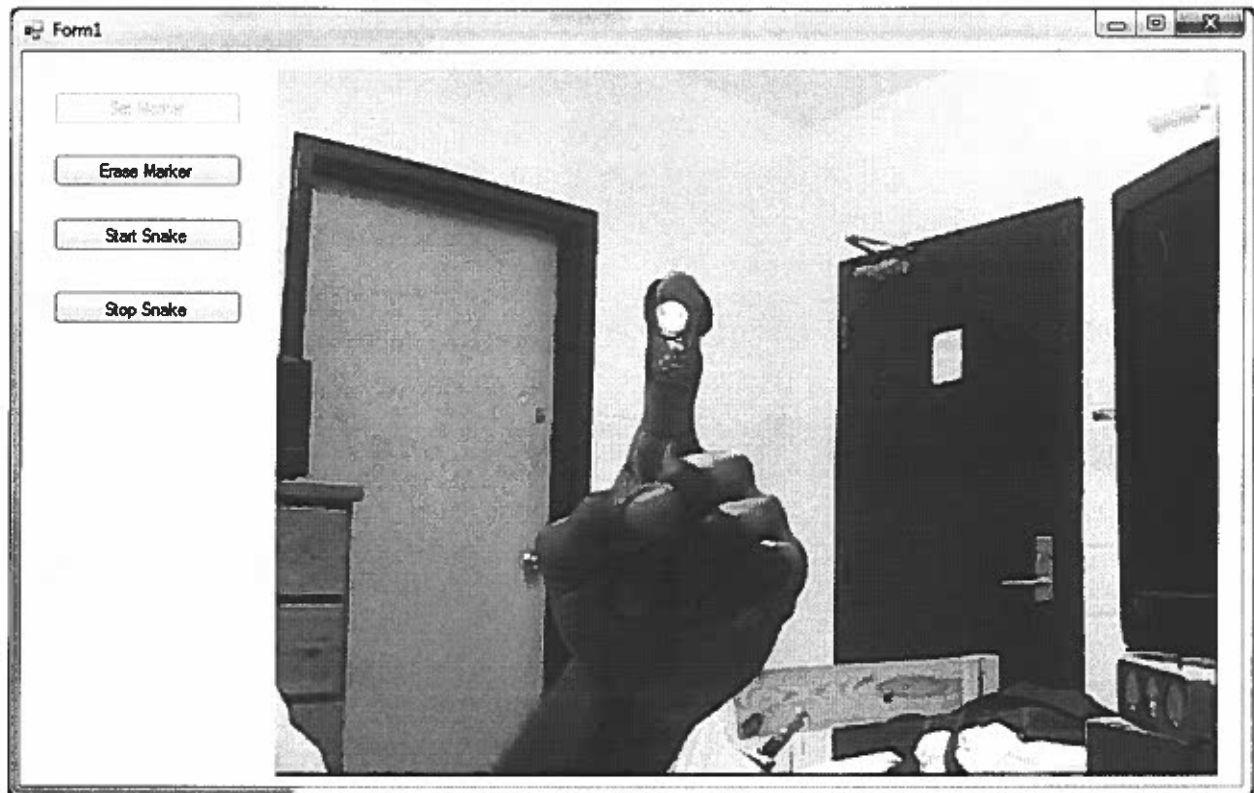
How to play Snake game

Mark Frost

1. When the program starts up initially, you are presented with a screen showing the video from your webcam along with a few options to choose from on the side of the video. The screen should look like this initially:

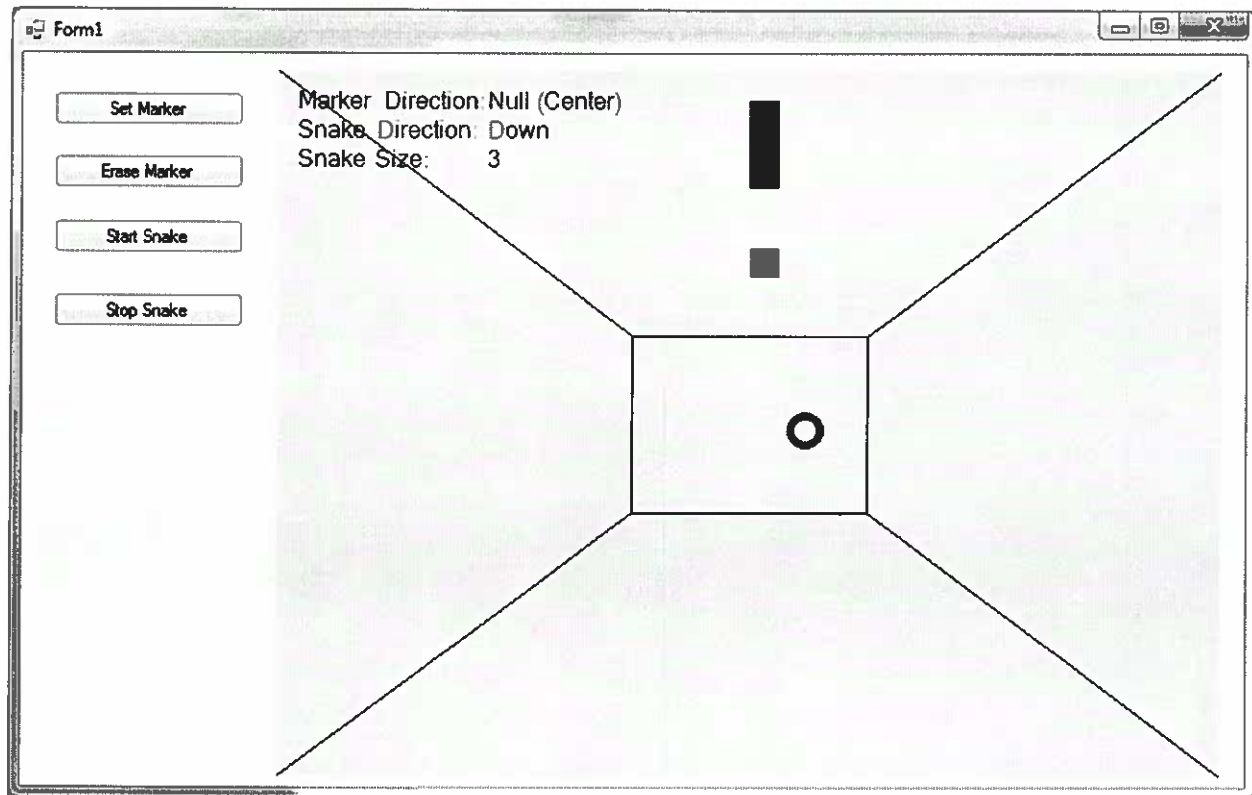


2. For the next step, you are going to want to place the object you intend to use as your marker in front of your webcam so we can set it. Once it is in front of your camera press the 'Set Marker' button, this will cause the image from the webcam to freeze allowing you to set your marker. To set the marker accurately, use the mouse to click and drag from the center of your marker until the radius of the circle encompasses a majority of the color of the object, and then release the mouse button.



Note: For improved accuracy tracking the marker, you will want to be in a well lit room and you will want your marker to be a color contrasting any colors in the background. Be sure that the size of the circle is not larger than it needs to be.

3. Once set, you will notice a box tracking your marker, if you see this then your marker has been set and you are ready to play the game. Now click on 'Start Snake' and you will be brought to this screen:



To play the game, you need to move your marker around the screen to make snake travel in a direction, the blue circle represents the position of your marker on the screen. You must make snake travel in the direction of the green apple by moving your marker in the direction that you want snake to travel. Snake can only turn; snake cannot go backwards; so if snake is traveling down, then it cannot go up when you bring your marker up.

Good luck and enjoy!

Touchless Implementation Code

Mark Frost

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using TouchlessLib;//the touchless library
using System.Drawing.Imaging;//for bitmap functionality
using System.Threading;

namespace TouchlessImplementation
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            TouchlessMgr _touch = new TouchlessMgr();//create a touchless instance
            Bitmap _overlay;//this is for the pixels we change later
            private static float RADIUS = 3; //used for default radius values
            private static float _markerRadius;
            private Point markerCenter;
            private Point _mouseOffset;
            private bool _drawRadius = false; //activates drawing of circle when selecting marker
            public bool addMarker = false; //lets us keep tracking whether we're adding a marker
            Thread mouseMover; //thread used to move mouse
            public int SENSISTIVITY;
            public bool relativity = true; //True = relative movement, false = absolute movement
            public bool flipImage = false; //flips the X axis of video image

            //*****Mouse Click Stuff*****
            [System.Runtime.InteropServices.DllImport("user32.dll")]
            public static extern void mouse_event(int dwFlags, int dx, int dy, int cButtons, int dwExtraInfo);

            public const int MOUSEEVENTF_LEFTDOWN = 0x02;
            public const int MOUSEEVENTF_LEFTUP = 0x04;
            public const int MOUSEEVENTF_RIGHTDOWN = 0x08;
            public const int MOUSEEVENTF_RIGHTUP = 0x10;
            //*****End Mouse Click Stuff*****

            new private void MouseClick()
            {
                int x = 100;
                int y = 100;

                mouse_event(MOUSEEVENTF_LEFTDOWN, x, y, 0, 0);
                mouse_event(MOUSEEVENTF_LEFTUP, x, y, 0, 0);
            }

            private void MouseRightClick()
            {
                int x = 100;
                int y = 100;

                mouse_event(MOUSEEVENTF_RIGHTDOWN, x, y, 0, 0);
                mouse_event(MOUSEEVENTF_RIGHTUP, x, y, 0, 0);
            }

            private void Form1_Load(object sender, EventArgs e)
            {

```

```

foreach (Camera c in _touch.Cameras)//for every camera that touchless found
{
    if (c != null)//check if its working
    {
        _touch.CurrentCamera = c;//set the current camera to the camera we found
        c.OnImageCaptured += new EventHandler<CameraEventArgs>(c_OnImageCaptured);//an event
handler to fire every time the new webcam gets an image
        _overlay = new Bitmap(c.CaptureWidth, c.CaptureHeight, PixelFormat.Format24bppRgb);//
use the bitmap constructor to set width, height, and 24bit RGB pixles
        _overlay.MakeTransparent();//makes the overlay transparent very easily!
        break;//break because we are only using one camera
    }
}
pictureBox1.Paint += new PaintEventHandler(pictureBox1_Paint);//created the paint event
mouseMover = new Thread(moveMouse); //thread used to move mouse
SENSISTIVITY = 1;
comboBox1.SelectedIndex = 1; //Keep sensitivity @ 1 by default
radioButton1.Checked = true; //Mouse Marker
radioButton2.Enabled = false; //Click Marker
}

void pictureBox1_Paint(object sender, PaintEventArgs e)
{
    lock (this)//
    {
        if (_b != null)//if the image _b has been captured
        {
            e.Graphics.DrawImageUnscaledAndClipped(_b, pictureBox1.ClientRectangle);//draw it on
the picture box
            e.Graphics.DrawImageUnscaledAndClipped(_overlay, pictureBox1.ClientRectangle);//draw
the overlay in the pictureBox
            if (_drawRadius)
            e.Graphics.DrawEllipse(new Pen(Brushes.Red, 2), markerCenter.X - _markerRadius,
markerCenter.Y - _markerRadius, 2 * _markerRadius, 2 * _markerRadius);
        }
    }
}

Bitmap _b;//a variable to store the new image we are going to get from the camera
void c_OnImageCaptured(object sender, CameraEventArgs e)
{
    if (button1.Enabled)//make sure the user isn't trying to set a marker
    {
        _b = e.Image;//save the image captured to _b
        if (flipImage) _b.RotateFlip(RotateFlipType.RotateNoneFlipX); //Mirrors the image in the
pictureBox
        else _b.RotateFlip(RotateFlipType.RotateNoneFlipNone); //Returns the image back to normal
        pictureBox1.Invalidate();//this causes the pictureBox to re paint (calling the paint event)
    }
}

//Set marker
private void button1_Click(object sender, EventArgs e)
{
    Bitmap c = _touch.CurrentCamera.GetCurrentImage();//save the image currently on the camera
    pictureBox1.BackgroundImage = c;//set that image to the background of pictureBox1
    button1.Enabled = false; //disable the button so that it can't be clicked again to crash the
program
    addMarker = true; //bool set so that the pictureBox1 allows you to click and drag to create a
marker
    //instead of being able to do it anytime which would then cause the program to fail b/c it
would try to
    //use an image that didn't exist anymore to set a marker.
}

```

```

//not used by default if MouseDown and MouseUp events are used
private void pictureBox1_MouseClick(object sender, MouseEventArgs e) //may use for default values
{
    if (radioButton1.Checked)
    {
        Marker m = _touch.AddMarker("marker", (Bitmap)pictureBox1.BackgroundImage, e.Location,
RADIUS);
        //created a new marker M using the background image that we saved and froze (typecasted to
bitmap), give it the point where the mouse is
        //and made it a hardcoded 5 pixles in size.
        m.Highlight = true; //this surrounds the maker with a box when we run
        m.SmoothingEnabled = true;
        m.Threshold = 2;
        m.OnChange += new EventHandler<MarkerEventArgs>(m_OnChange); // create a new event for when
the marker changes
        button1.Enabled = true; //re enable the button because we are now done makeing a marker
        radioButton2.Enabled = true;
    }
    else if (radioButton2.Checked)
    {
        Marker n = _touch.AddMarker("nmarker", (Bitmap)pictureBox1.BackgroundImage, e.Location,
RADIUS);
        //created a new marker M using the background image that we saved and froze (typecasted to
bitmap), give it the point where the mouse is
        //and made it a hardcoded 5 pixles in size.
        n.Highlight = true; //this surrounds the maker with a box when we run
        n.SmoothingEnabled = true;
        n.Threshold = 2;
        n.OnChange += new EventHandler<MarkerEventArgs>(n_OnChange); // create a new event for when
the marker changes
        button1.Enabled = true; //re enable the button because we are now done makeing a marker
    }
}

//Set center point for setting radius
private void pictureBox1_MouseDown(object sender, MouseEventArgs e)
{
    if (addMarker)
    {
        markerCenter = e.Location; //sets the center of the marker to help determine the radius
when dragging.
        _markerRadius = 0;

        // Begin drawing the selection adornment
        _drawRadius = true;
    }
}

//Setting marker and adding it to _touch (touchless manager)
private void pictureBox1_MouseUp(object sender, MouseEventArgs e)
{
    if (addMarker==false) {}
    else {
        if (radioButton1.Checked)
        {
            Marker m = _touch.AddMarker("marker", (Bitmap)pictureBox1.BackgroundImage, e.Location,
_markerRadius);
            //created a new marker M using the background image that we saved and froze (typecasted
to bitmap), give it the point where the mouse is
            m.Highlight = true; //this surrounds the maker with a box when we run
            m.SmoothingEnabled = true; //rounds out some of the x,y data from the raw data from the
processed image
            m.Threshold = 2; //Helps with the color recongnition, if it's too high it's hard to
keep track of it / too low then it tracks too many objects.

```

```

        m.OnChange += new EventHandler<MarkerEventArgs>(m_OnChange); // create a new event for ✓
when the marker changes
        button1.Enabled = true; //re enable the button because we are now done making a marker
        radioButton2.Enabled = true;
        addMarker = false;
    }
    else if (radioButton2.Checked)
    {
        Marker n = _touch.AddMarker("nmarker", (Bitmap)pictureBox1.BackgroundImage, e.Location, ✓
_markerRadius);
        //created a new marker N using the background image that we saved and froze (typecasted ✓
to bitmap), give it the point where the mouse is
        n.Highlight = true; //this surrounds the maker with a box when we run
        n.SmoothingEnabled = true;
        n.Threshold = 2;
        n.OnChange += new EventHandler<MarkerEventArgs>(n_OnChange); // create a new event for ✓
when the marker changes
        button1.Enabled = true; //re enable the button because we are now done making a marker
        addMarker = false;
    }
    _drawRadius = false;
}

//Drawing radius when setting marker
private void pictureBox1_MouseMove(object sender, MouseEventArgs e)
{
    if (addMarker && !markerCenter.IsEmpty)
    {
        // Get the current radius from center point when the mouse was clicked
        int dx = e.X - markerCenter.X;
        int dy = e.Y - markerCenter.Y;
        _markerRadius = (float)Math.Sqrt(dx * dx + dy * dy);

        // Cause display Update
        pictureBox1.Invalidate();
    }
}

//Clicker
void n_OnChange(object sender, MarkerEventArgs e)
{
    if (e.EventData.Present && !e.EventMarker.PreviousData.Present) //notes when the marker just ✓
became visable
    {
        //Basically what this does is that if the 1st(mouse) marker is close enough to the screen ✓
and the
        //click marker comes up, then instead of a regular click, it will perform a right click ✓
instead.
        //May be able to be implemented better somehow...?
        if (_touch.Markers.ElementAt(0).CurrentData.Bounds.Height > (pictureBox1.Height / 4))
            MouseRightClick();
        else
            MouseClick();
    }
}

void m_OnChange(object sender, MarkerEventArgs e)
{
    int steps = 10;
    int DX = e.EventMarker.PreviousData.X - e.EventData.X;
    int DY = e.EventMarker.PreviousData.Y - e.EventData.Y;
    //makes sure the marker is in the image and the change is position is great enough
    //to warrant the cursor to move.
    if (e.EventData.Present /*&& (DX > .1 || DX < -.1) && (DY > .1 || DY < -.1)*/)
    {

```

```

        //mouseMover.Start(e);
        PointF iterPoint = Cursor.Position;
        PointF newpos;
        if (!relativity) //absolute pointer
        {
            //Calculates X,Y in relation to the scaled sizes between main window and the form window
            newpos = new PointF((float)(System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width - (e
            .EventData.X * (System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / pictureBox1.Width))*1.2),
            (float)(e.EventData.Y * (System.Windows.Forms.Screen.PrimaryScreen.Bounds.
            Height / (pictureBox1.Height - 60))));
        }
        else //relative pointer
        {
            //Uses the Change in X,Y (DX,DY) to determine movement distance
            newpos = new PointF((float)(Cursor.Position.X - (e.EventData.DX * (System.Windows.Forms
            .Screen.PrimaryScreen.Bounds.Width / pictureBox1.Width))),
            (float)(Cursor.Position.Y + (e.EventData.DY * (System.Windows.Forms.Screen.
            PrimaryScreen.Bounds.Height / pictureBox1.Height))));
        }

        //Difference of new point and current point to determine the direction and distance to move
        PointF slope = new PointF(newpos.X - Cursor.Position.X, newpos.Y - Cursor.Position.Y);

        new Thread(() =>
        {
            slope.X = (slope.X * SENSISTIVITY) / steps; //X & Y coordinates that are added each
            slope.Y = (slope.Y * SENSISTIVITY) / steps; //to move the cursor in a linear direction
            for (int i = 0; i < steps; i++)
            {
                //New point for cursor to jump to
                iterPoint = new PointF(iterPoint.X + slope.X, iterPoint.Y + slope.Y);
                Cursor.Position = (Point.Round(iterPoint)); //Send cursor X & Y of next point
                Thread.Sleep(5);
            }
        }).Start();
    }
    //This is usable if you want the 'mouse click' to be if the marker is close to the screen or
    light is shined directly into the camera
    //else if (e.EventData.Present && (e.EventMarker.CurrentData.Bounds.Height > (pictureBox1.
    Height / 4)))
    //{
        //    MouseClick();
        //    Thread.Sleep(50);
    //}

    /*if (!relativity && e.EventData.Present)
    {
        PointF iterPoint = Cursor.Position;
        PointF newpos = new PointF(System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width - (e.
        EventData.X * (System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / (pictureBox1.Width - 40))),
        e.EventData.Y * (System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height /
        (pictureBox1.Height - 60)));

        PointF slope = new PointF(newpos.X - Cursor.Position.X, newpos.Y - Cursor.Position.Y);

        new Thread(() =>
        {
            slope.X = (slope.X * SENSISTIVITY) / steps;
            slope.Y = (slope.Y * SENSISTIVITY) / steps;
            for (int i = 0; i < steps; i++)
            {
                iterPoint = new PointF(iterPoint.X + slope.X, iterPoint.Y + slope.Y);
                Cursor.Position = (Point.Round(iterPoint));
                Thread.Sleep(5);
            }
        })
    }

```

```

        ).Start();

        lock (this)//make sure we don't paint at the same time we are changing
        {
            this is where we can do stuff every time the marker changes!!
            int curX = System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width - e.EventData.X * (System
.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / pictureBox1.Width);
            int curY = e.EventData.Y * (System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height /
pictureBox1.Height);
            if (e.EventData.Present)//make sure the marker is currently being seen in the image
            {
                _overlay.SetPixel(e.EventData.X, e.EventData.Y, Color.Black);//change the pixel at
the new marker location to black

                Cursor.Position = new Point(System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width
- (e.EventData.X * (System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / (pictureBox1.Width - 40)))
,
                e.EventData.Y * (System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height /
(pictureBox1.Height - 60)));
                Cursor.Position = new Point(
                if ((e.EventData.X > e.EventMarker.LastGoodData.X + 3) || (e.EventData.X < e.
EventMarker.LastGoodData.X - 5))//maker went right
                {
                    Cursor.Position = new Point(Cursor.Position.X + (e.EventMarker.LastGoodData.X -
e.EventData.X)*3 , Cursor.Position.Y);
                }
                if ((e.EventData.Y > e.EventMarker.LastGoodData.Y + 3) || (e.EventData.Y < e.
EventMarker.LastGoodData.Y - 5))//maker went down
                {
                    Cursor.Position = new Point(Cursor.Position.X, Cursor.Position.Y + (e.EventData
.Y - e.EventMarker.LastGoodData.Y) * 3);
                }
            }
        }
    }*/
}

private void moveMouse(object temp) //original mouse movement function, to move mouse only;
relative/absolute handled in m_onchange function
{
    MarkerEventArgs e = (MarkerEventArgs)temp;
    int steps = 10;
    PointF iterPoint = Cursor.Position;
    PointF newPos = new PointF((float)(Cursor.Position.X - (e.EventData.DX * (System.Windows.Forms.
Screen.PrimaryScreen.Bounds.Width / pictureBox1.Width))),
    (float)(Cursor.Position.Y + (e.EventData.DY * (System.Windows.Forms.Screen.PrimaryScreen.
Bounds.Height / pictureBox1.Height))));

    PointF slope = new PointF(newPos.X - Cursor.Position.X, newPos.Y - Cursor.Position.Y);

    slope.X = slope.X / steps;
    slope.Y = slope.Y / steps;
    for (int i = 0; i < steps; i++)
    {
        iterPoint = new PointF(iterPoint.X + slope.X, iterPoint.Y + slope.Y);
        Cursor.Position = (Point.Round(iterPoint));
        Thread.Sleep(2);
    }
}

//Erase all markers
private void button2_Click(object sender, EventArgs e)
{
    for (int i = _touch.MarkerCount-1; i >= 0; i--)
    {
        _touch.RemoveMarker(i);
    }
}

```

```

    }
    radioButton1.Select();
    radioButton2.Enabled = false;
}

//Erase specific marker
private void button3_Click(object sender, EventArgs e)
{
    if (radioButton1.Checked)
    {
        for (int i = 0; i < _touch.MarkerCount; i++)
        {
            if (_touch.Markers.ElementAt(i).Name == "marker")
                _touch.RemoveMarker(i);
        }
    }
    else if (radioButton2.Checked)
    {
        for (int i = 0; i < _touch.MarkerCount; i++)
        {
            if (_touch.Markers.ElementAt(i).Name == "nmarker")
                _touch.RemoveMarker(i);
        }
    }
    else { }
}

//Sensitivity
private void numericUpDown1_ValueChanged(object sender, EventArgs e)
{
    SENSISTIVITY = (int)numericUpDown1.Value; //affects the mouse movement calculation
}

//Relative | Absolute
private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    if (comboBox1.SelectedIndex == 0)
    {
        relativity = true; //sets marker in relative mode
        numericUpDown1.Enabled = true; //enables the sensitivity box
    }
    else
    {
        relativity = false; //sets marker in absolute mode
        numericUpDown1.Enabled = false;
    }
}

private void Form1_FormClosing(object sender, FormClosingEventArgs e)
{
    _touch.Dispose();
}

//Mirror the image in the picturebox
private void button4_Click(object sender, EventArgs e)
{
    if (!flipImage) button4.Text = "Switch Back to Normal";
    else button4.Text = "Flip Image";
    flipImage = !flipImage; //bool switch for when an image is caputured from the webcam
}
}
}

```

Snake Game Demo Code

Mark Frost

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using TouchlessLib;//the touchless library
using System.Drawing.Imaging;//for bitmap functionality
using System.Threading;

namespace SnakeGame
{
    public partial class SnakeGame : Form
    {
        public SnakeGame()
        {
            InitializeComponent();

            TouchlessMgr _touch = new TouchlessMgr();//create a touchless instance
            Bitmap _overlay;//this is for the pixels we change later
            private static float _markerRadius;
            private Point markerCenter;
            private bool _drawRadius = false; //activates drawing of circle when selecting marker
            public bool addMarker = false; //lets us keep tracking whether we're adding a marker
            public int SENSISTIVITY;
            public bool relativity = true; //True = relative movement, false = absolute movement
            IDemoInterface demo = new SnakeDemo();
            bool playing = false;
            public PointF relativeCam;

            private void Form1_Load(object sender, EventArgs e)
            {
                foreach (Camera c in _touch.Cameras)//for every camera that touchless found
                {
                    if (c != null)//check if its working
                    {
                        relativeCam.X = c.CaptureWidth;
                        relativeCam.Y = c.CaptureHeight;
                        _touch.CurrentCamera = c;//set the current camera to the camera we found
                        c.OnImageCaptured += new EventHandler<CameraEventArgs>(c_OnImageCaptured);//an event
handler to fire every time the new webcam gets an image
                        _overlay = new Bitmap(c.CaptureWidth, c.CaptureHeight, PixelFormat.Format24bppRgb);//
use the bitmap constructor to set width, height, and 24bit RGB pixles
                        _overlay.MakeTransparent();//makes the overlay transparent very easily!
                        break;//break becasue we are only using one camera
                    }
                }
                pictureBox1.Paint += new PaintEventHandler(pictureBox1_Paint);//created the paint event
                SENSISTIVITY = 1;
                relativeCam.X = pictureBox1.Size.Width / relativeCam.X;
                relativeCam.Y = pictureBox1.Size.Height / relativeCam.Y;
            }

            void pictureBox1_Paint(object sender, PaintEventArgs e)
            {
                lock (this)//
                {
                    if (_b != null)//if the image _b has been captured
                    {
                        if (playing)

```

```

        {
            demo.DrawDemoCanvas(e.Graphics);
        }
        else
        {
            e.Graphics.DrawImageUnscaledAndClipped(_b, pictureBox1.ClientRectangle); //draw it ✓
on the picture box
            e.Graphics.DrawImageUnscaledAndClipped(_overlay, pictureBox1.ClientRectangle); // ✓
draw the overlay in the picturebox
            if (_drawRadius)
                e.Graphics.DrawEllipse(new Pen(Brushes.Red, 2), markerCenter.X - _markerRadius, ✓
markerCenter.Y - _markerRadius, 2 * _markerRadius, 2 * _markerRadius);
        }
    }
}

Bitmap _b; //a variable to store the new image we are going to get from the camera
void c_OnImageCaptured(object sender, CameraEventArgs e)
{
    if (button1.Enabled) //make sure the user isn't trying to set a marker
    {
        _b = e.Image; //save the image captured to _b
        _b.RotateFlip(RotateFlipType.RotateNoneFlipX); //flips X axis for image
        pictureBox1.Invalidate(); //this causes the picturebox to re paint (calling the paint event)
    }
}

private void button1_Click(object sender, EventArgs e)
{
    Bitmap c = _touch.CurrentCamera.GetCurrentImage(); //save the image currently on the camera
    pictureBox1.BackgroundImage = c; //set that image to the background of pictureBox1
    button1.Enabled = false;
    addMarker = true;
}

private void pictureBox1_MouseDown(object sender, MouseEventArgs e)
{
    if (addMarker)
    {
        markerCenter = e.Location;
        _markerRadius = 0;

        // Begin drawing the selection adornment
        _drawRadius = true;
    }
}

private void pictureBox1_MouseUp(object sender, MouseEventArgs e)
{
    if (addMarker == false) { }
    else
    {
        Point scaleCam = new Point(e.Location.X / (int)relativeCam.X, e.Location.Y / (int) ✓
relativeCam.Y);
        _markerRadius = (_markerRadius * _b.Height) / pictureBox1.Height;
        Marker m = _touch.AddMarker("Marker #1", (Bitmap)pictureBox1.BackgroundImage, scaleCam, ✓
        _markerRadius);
        //created a new marker M using the background image that we saved and froze (typecasted to ✓
        bitmap), give it the point where the mouse is
        //and made it a hardcoded 5 pixles in size.
        m.Highlight = true; //this surrounds the maker with a box when we run
        m.SmoothingEnabled = true;
        m.Threshold = 2;
    }
}

```

```

        //m.OnChange += new EventHandler<MarkerEventArgs>(m_OnChange); // create a new event for
when the marker changes
        button1.Enabled = true; //re enable the button because we are now done making a marker
        addMarker = false;

        _drawRadius = false;
    }
}

private void pictureBox1_MouseMove(object sender, MouseEventArgs e)
{
    if (addMarker && !markerCenter.IsEmpty)
    {
        // Get the current radius
        int dx = e.X - markerCenter.X;
        int dy = e.Y - markerCenter.Y;
        _markerRadius = (float)Math.Sqrt(dx * dx + dy * dy);

        // Cause display Update
        pictureBox1.Invalidate();
    }
}

void m_OnChange(object sender, MarkerEventArgs e)
{
    int steps = 10;
    int DX = e.EventMarker.PreviousData.X - e.EventData.X;
    int DY = e.EventMarker.PreviousData.Y - e.EventData.Y;
    //makes sure the marker is in the image and the change is position is great enough
    //to warrant the cursor to move.
    if (e.EventData.Present /*&& (DX > .1 || DX < -.1) && (DY > .1 || DY < -.1)*/)
    {
        //mouseMover.Start(e);
        PointF iterPoint = Cursor.Position;
        PointF newpos;
        if (!relativity) //absolute pointer
        {
            //Calculates X,Y in relation to the scaled sizes between main window and the form window
            newpos = new PointF((float)(System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width - (e
.EventData.X * (System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / pictureBox1.Width)) * 1.2),
(float)(e.EventData.Y * (System.Windows.Forms.Screen.PrimaryScreen.Bounds.
Height / (pictureBox1.Height - 60))));
        }
        else //relative pointer
        {
            //Uses the Change in X,Y (DX,DY) to determine movement distance
            newpos = new PointF((float)(Cursor.Position.X - (e.EventData.DX * (System.Windows.Forms
.Screen.PrimaryScreen.Bounds.Width / pictureBox1.Width))),
(float)(Cursor.Position.Y + (e.EventData.DY * (System.Windows.Forms.Screen.
PrimaryScreen.Bounds.Height / pictureBox1.Height))));
        }

        //Difference of new point and current point to determine the direction and distance to move
        PointF slope = new PointF(newpos.X - Cursor.Position.X, newpos.Y - Cursor.Position.Y);

        new Thread(() =>
        {
            slope.X = (slope.X * SENSISTIVITY) / steps; //X & Y coordinates that are added each
            slope.Y = (slope.Y * SENSISTIVITY) / steps; //to move the cursor in a linear direction
            for (int i = 0; i < steps; i++)
            {
                //New point for cursor to jump to
                iterPoint = new PointF(iterPoint.X + slope.X, iterPoint.Y + slope.Y);
                Cursor.Position = (Point.Round(iterPoint)); //Send cursor X & Y of next point
                Thread.Sleep(5);
            }
        })
    }
}

```

```
        ).Start();
    }
}

private void button2_Click(object sender, EventArgs e)
{
    for (int i = _touch.MarkerCount - 1; i >= 0; i--)
    {
        _touch.RemoveMarker(i);
    }
}

private void button3_Click(object sender, EventArgs e)
{
    if (demo.StartDemo(_touch, pictureBox1.Bounds)) //ensures we aren't trying to draw the game
when it hasn't even started                                ✓
    playing = true;
}

private void button4_Click(object sender, EventArgs e)
{
    demo.StopDemo();
    playing = false;
}

private void SnakeGame_FormClosing(object sender, FormClosingEventArgs e)
{
    _touch.Dispose();
}
}

public interface IDemoInterface
{
    string GetDemoDescription();
    bool StartDemo(TouchlessMgr tlmgr, Rectangle displayBounds);
    void StopDemo();
    void DrawDemoCanvas(Graphics gfx);
}
}
```

```

//*****
// File:      SnakeDemo.cs
// Project:    TouchlessDemo
// Author(s):  Michael Wasserman (Michael.Wasserman@microsoft.com)
//
// Demo to play a simple version of the classic game, snake.
//*****

```

```

using System;
using System.Collections.Generic;
using System.Drawing;
using System.Windows.Forms;
using TouchlessLib;

```

```

namespace SnakeGame

```

```

{
    public class SnakeDemo : IDemoInterface
    {
        public override string ToString() { return "Snake Demo"; }

```

```

        #region IDemoInterface

```

```

        string IDemoInterface.GetDemoDescription()
        {

```

```

            return

```

```

@"Snake Demo Instructions:

```

Uses a single marker to control this classic snake game remake.

> Use the four areas (up/down/left/right) to direct the snake.

> The center area doesn't effect the snake direction.

> The snake won't double-back, turn 90 degrees first.

Collect as many little nibbles as you can to increase your score.

Watch out: the snake will grow and get faster.

Don't hit the walls and don't bite yourself.

Can you make more snake levels with obstacles?

Can you revise the snake's movement and controls?

Can you remake other classic games or invent a new game?

Our SDK isn't tied to Windows Forms, use XNA and more.

Give feedback, submit code, join the community, and more:

<http://www.codeplex.com/touchless>;

```

    }

```

```

    bool IDemoInterface.StartDemo(TouchlessMgr tlmgr, Rectangle displayBounds)
    {

```

```

        if (tlmgr.MarkerCount < 1)
        {

```

```

            MessageBox.Show("Snake Demo requires a marker.", "Warning", MessageBoxButtons.OK,

```

```

            MessageBoxIcon.Warning);

```

```

            return false;

```

```

        }

```

```

        _tlmgr = tlmgr;

```

```

        _captureWidth = tlmgr.CurrentCamera.CaptureWidth;

```

```

        _captureHeight = tlmgr.CurrentCamera.CaptureHeight;

```

```

        _displayScale = displayBounds.Width / _captureWidth;

```

```

        _canvas = new Bitmap(displayBounds.Width, displayBounds.Height);

```

```

        _rows = 24;

```

```

        _cols = 32;

```

```

        _cellSize = new Size(_captureWidth * _displayScale / _cols, _captureHeight * _displayScale /

```

```

        _rows);

```

```

        _backgroundBrush = Brushes.Wheat;

```

```

        _collisionBrush = Brushes.Red;

```

```

        _nibbleBrush = Brushes.Green;
        _rand = new Random();
        _updatingSnake = false;

        // Make a new snake
        _snake = new Snake(Brushes.Black);
        _snake.body.Insert(0, new Point(_captureWidth * _displayScale / 2, 0));
        _snake.body.Insert(0, new Point(_captureWidth * _displayScale / 2, _cellSize.Height));
        _snake.body.Insert(0, new Point(_captureWidth * _displayScale / 2, _cellSize.Height * 2));
        _snake.dx = _snake.dxm = 0;
        _snake.dy = _snake.dym = 1;
        _lastStateUpdate = DateTime.Now;

        _hudStringMarkerDir = "Null (Center)";
        _hudStringSnakeDir = "Down";
        _hudStringSnakeSize = _snake.body.Count.ToString();

        // Clear the canvas object using the background brush
        Graphics gfx = Graphics.FromImage(_canvas);
        gfx.FillRectangle(_backgroundBrush, 0, 0, _captureWidth * _displayScale, _captureHeight *
_displayScale);
        // Draw the initial snake
        Rectangle snakeRect = new Rectangle(_snake.body[2], new Size(_cellSize.Width, 3 * _cellSize.
Height));
        gfx.FillRectangle(_snake.brush, snakeRect);
        // Make and draw the first nibble
        _nibble = new Point(_captureWidth * _displayScale / 2, _cellSize.Height * 6);
        Rectangle nibbleRect = new Rectangle(_nibble, _cellSize);
        gfx.FillRectangle(_nibbleBrush, nibbleRect);

        // Create a rectangle that represents the center-origin rect for null input
        _nullZone = new Rectangle(-_captureWidth / 8, -_captureHeight / 8, _captureWidth / 4,
_captureHeight / 4);
        _nullZoneDisplay = new Rectangle(_captureWidth / 2 - _captureWidth / 8, _captureHeight / 2 -
_captureHeight / 8, _captureWidth / 4, _captureHeight / 4);
        // Precalculate the corner angles for input partitioning
        _angleNW = (int)(Math.Atan2(_captureHeight / 2, -_captureWidth / 2) * 180 / Math.PI);
        _angleNE = (int)(Math.Atan2(_captureHeight / 2, _captureWidth / 2) * 180 / Math.PI);
        _angleSE = (int)(Math.Atan2(-_captureHeight / 2, _captureWidth / 2) * 180 / Math.PI);
        _angleSW = (int)(Math.Atan2(-_captureHeight / 2, -_captureWidth / 2) * 180 / Math.PI);

        // Add marker update handling
        _tlmgr.Markers[0].OnChange += new EventHandler<MarkerEventArgs>(UpdateMarker);

        return true;
    }

    void IDemoInterface.StopDemo()
    {
        if (_tlmgr == null || _tlmgr.Markers == null)
            return;

        // Remove marker update handling
        _tlmgr.Markers[0].OnChange -= new EventHandler<MarkerEventArgs>(UpdateMarker);

        _tlmgr = null;
    }

    void IDemoInterface.DrawDemoCanvas(Graphics gfx)
    {
        if (_tlmgr == null)
            return;

        // should we Update the snake's state?
        if (!_snake.collision && (DateTime.Now.Ticks - _lastStateUpdate.Ticks) / TimeSpan.
TicksPerMillisecond > _snake.updateMS)

```

```

    {
        UpdateSnake();
        _lastStateUpdate = DateTime.Now;
    }
    // Draw the game image
    gfx.DrawImage(_canvas, 0, 0);
    // Draw the marker location
    gfx.DrawEllipse(new Pen(Color.Blue, 5), _xm * _displayScale - 10, _ym * _displayScale - 10, 20, 20);
    // Draw the marker direction divisions
    gfx.DrawRectangle(Pens.Blue, _nullZoneDisplay.Left * _displayScale, _nullZoneDisplay.Top * _displayScale, _nullZoneDisplay.Width * _displayScale, _nullZoneDisplay.Height * _displayScale);
    gfx.DrawLine(Pens.Blue, _nullZoneDisplay.Left * _displayScale, _nullZoneDisplay.Top * _displayScale, 0, 0);
    gfx.DrawLine(Pens.Blue, _nullZoneDisplay.Right * _displayScale, _nullZoneDisplay.Top * _displayScale, _captureWidth * _displayScale, 0);
    gfx.DrawLine(Pens.Blue, _nullZoneDisplay.Right * _displayScale, _nullZoneDisplay.Bottom * _displayScale, _captureWidth * _displayScale, _captureHeight * _displayScale);
    gfx.DrawLine(Pens.Blue, _nullZoneDisplay.Left * _displayScale, _nullZoneDisplay.Bottom * _displayScale, 0, _captureHeight * _displayScale);
    // Draw the hud
    gfx.DrawString("Marker Direction:\t" + _hudStringMarkerDir, new Font(FontFamily.GenericSansSerif, 12), Brushes.Brown, new PointF(10, 10));
    gfx.DrawString("Snake Direction:\t" + _hudStringSnakeDir, new Font(FontFamily.GenericSansSerif, 12), Brushes.Brown, new PointF(10, 30));
    gfx.DrawString("Snake Size:\t" + _hudStringSnakeSize, new Font(FontFamily.GenericSansSerif, 12), Brushes.Brown, new PointF(10, 50));
    if (_snake.collision)
        gfx.DrawString("GAME OVER", new Font(FontFamily.GenericSansSerif, 30), Brushes.Brown, new PointF(_captureWidth * _displayScale / 2 - 130, _captureHeight * _displayScale / 2 - 25));
}

#endregion IDemoInterface

private void UpdateSnake()
{
    if (_updatingSnake)
        return;

    _updatingSnake = true;

    Graphics gfx = Graphics.FromImage(_canvas);

    // Update the actual snake direction with the latest marker direction (prevent double-backing and null setting)
    if ((_snake.dxm != -_snake.dx || _snake.dx == 0) && (_snake.dym != -_snake.dy || _snake.dy == 0) && _snake.dxm != _snake.dym)
    {
        _hudStringSnakeDir = _hudStringMarkerDir;
        _snake.dx = _snake.dxm;
        _snake.dy = _snake.dym;
    }

    // Get the new head point
    Point headPoint = new Point(_snake.body[0].X, _snake.body[0].Y);
    headPoint.X += _cellSize.Width * _snake.dx;
    headPoint.Y += _cellSize.Height * _snake.dy;

    // Check if we've eaten the nibble
    if (!Point.Equals(headPoint, _nibble))
    {
        // Erase the tail piece
        Point tailPoint = _snake.body[_snake.body.Count - 1];
        Rectangle tailRect = new Rectangle(tailPoint, _cellSize);
        gfx.FillRectangle(_backgroundBrush, tailRect);
        _snake.body.RemoveAt(_snake.body.Count - 1);
    }
}

```

```

    }
    else
    {
        // Make a new nibble, not on the snake
        bool nibbleClear = true;
        do
        {
            _nibble = new Point(_rand.Next(_cols) * _cellSize.Width, _rand.Next(_rows) * _cellSize.
Height);
            Point[] body = new Point[_snake.body.Count];
            _snake.body.CopyTo(0, body, 0, body.Length);
            nibbleClear = true;
            for (int i = 0; i < body.Length; i++)
                if (Point.Equals(_nibble, body[i]))
                {
                    nibbleClear = false;
                    break;
                }
            _snake.updateMS = (int)(_snake.updateMS * _snake.updateMSMultiplier);
        } while (!nibbleClear);
        Rectangle nibbleRect = new Rectangle(_nibble, _cellSize);
        gfx.FillRectangle(_nibbleBrush, nibbleRect);
    }

    // Check if we've collided with a wall or ourself
    if (headPoint.X < 0 || headPoint.X > _captureWidth * _displayScale - _cellSize.Width ||
        headPoint.Y < 0 || headPoint.Y > _captureHeight * _displayScale - _cellSize.Height)
        _snake.collision = true;
    else
    {
        Point[] body = new Point[_snake.body.Count];
        _snake.body.CopyTo(0, body, 0, body.Length);
        for (int i = 0; i < body.Length; i++)
            if (Point.Equals(headPoint, body[i]))
                _snake.collision = true;
    }

    // Add the new head piece
    Rectangle headRect = new Rectangle(headPoint, _cellSize);
    gfx.FillRectangle((_snake.collision) ? _collisionBrush : _snake.brush, headRect);
    _snake.body.Insert(0, headPoint);

    _hudStringSnakeSize = _snake.body.Count.ToString();
    _updatingSnake = false;
}

private void UpdateMarker(object sender, MarkerEventArgs args)
{
    if (!args.EventData.Present)
        return;

    MarkerEventData data = args.EventData;
    _xm = (int)_captureWidth - data.X;
    _ym = (int)data.Y;

    // Translate x & y coords to have an origin of the image center, and flip Y
    int x = (int)(_captureWidth / 2 - data.X);
    int y = (int)(_captureHeight / 2 - data.Y);

    // Is the marker in our null region?
    if (_nullZone.Contains(new Point(x, y)))
    {
        _hudStringMarkerDir = "Null (Center)";
        _snake.dxm = _snake.dym = 0;
        return;
    }
}

```

```

// Get the angle of the point from the center
// angle: E = 0, W = 180(ccw)/-180(cw)
int angle = (int)(Math.Atan2(y, x) * 180 / Math.PI);

// Compare the angle with the diagonals and change the snake direction, ignore double-backing
if (angle < _angleSW || angle >= _angleNW)
{
    _hudStringMarkerDir = "Left";
    if (_snake.dx == 1)
        return;
    _snake.dxm = -1;
    _snake.dym = 0;
}
else if (angle < _angleSE)
{
    _hudStringMarkerDir = "Down";
    if (_snake.dy == -1)
        return;
    _snake.dxm = 0;
    _snake.dym = 1;
}
else if (angle < _angleNE)
{
    _hudStringMarkerDir = "Right";
    if (_snake.dx == -1)
        return;
    _snake.dxm = 1;
    _snake.dym = 0;
}
else /*if (angle < _angleNW)*/
{
    _hudStringMarkerDir = "Up";
    if (_snake.dy == 1)
        return;
    _snake.dxm = 0;
    _snake.dym = -1;
}
}

// Drawing and input
private TouchlessMgr _tlmgr;
private Bitmap _canvas;
private int _captureWidth, _captureHeight, _displayScale;
private string _hudStringMarkerDir, _hudStringSnakeDir, _hudStringSnakeSize;
private bool _updatingSnake;

// Input recognition
private DateTime _lastStateUpdate;
private Rectangle _nullZone, _nullZoneDisplay;
private int _angleNW, _angleNE, _angleSE, _angleSW;
// Marker position
private int _xm, _ym;

// Grid info
private byte _rows, _cols;
private Size _cellSize;
private Brush _backgroundBrush;
private Brush _collisionBrush;
private Brush _nibbleBrush;

// Snake info
private Snake _snake;
private Random _rand;
private Point _nibble;

```

```
/// <summary>
/// Structure to represent the snake
/// </summary>
private struct Snake
{
    public Snake(Brush b)
    {
        dx = dy = dxm = dym = 0;
        body = new List<Point>();
        brush = b;
        collision = false;
        updateMS = 500;
        updateMSMultiplier = 0.97;
    }

    // Snake direction per axis, updated once per step
    public short dx, dy;
    // Marker direction per axis
    public short dxm, dym;
    // Snake Point list that composes its body
    public List<Point> body;
    // The snake brush
    public Brush brush;
    // Has the snake collided?
    public bool collision;
    // The MS to wait between updates, and decrement value
    public int updateMS;
    // The multiplier applied to updateMS for each nibble
    public double updateMSMultiplier;
}
}
```

Touchless SDK Documentation

Homepage: <http://www.codeplex.com/touchless>

Project Author: Michael Wasserman

Michael.Wasserman@microsoft.com

Thanks due to: Gary Caldwell, Isabel Mattos, Nicole Steinbok, Quinn Hawkins, MS Office Labs Community Projects, John Kender, and you!

0. Table of contents

- 0. Table of contents (oh no! infinite loop!)
- 1. What's Touchless?
- 2. What's in this release?
- 3. What's in the source repository?
- 4. How to add Touchless to your project
- 5. Reference Information
- 6. Backlog of ideas

1. What's Touchless?

Touchless is an SDK that allows users to create and experience multi-touch applications. Touchless started as Mike Wasserman's college project at Columbia University. The main idea: to offer users a new and cheap way of experiencing multi-touch capabilities, without the need of expensive hardware or software. All the user needs is a camera, which will track colored markers defined by the user.

Mike presented the project at the Microsoft Office Labs Productivity Science Fair, Office Labs fell in love with it, and Touchless was chosen as a Community Project. Our deliverables include an extensible demo application to showcase a limited set of multi-touch capabilities, but mainly we are delivering an SDK to allow users to build their own multi-touch applications.

Now, Touchless is released free and open-source to the world under the Microsoft Public License (Ms-PL) on CodePlex. Our goals are to drive community involvement and use of the SDK as it continues to develop.

Remember that this is just the beginning; and you're invited to join our journey. Send us

your questions and feedback, use Touchless SDK in your .NET applications and XNA games, and support the community by contributing to the source code.

Unleash the power of your webcam!

2. What's in this release?

This release includes a short list of binary files to demonstrate Touchless:

- WebCamLib.dll - Interfaces with DirectShow to grab webcam frames
- TouchlessLib.dll - Contains the functionality of Touchless SDK
- TouchlessDemo.exe - A small demonstration of what Touchless can be used for
- Image.gif - Used in TouchlessDemo's image demo
- Touchless.chm - A documentation file of the Touchless API for developers
- Touchless.jpg - A class diagram of the Touchless API for developers
- Touchless.rtf - This file; you're reading it right now!

If you just want to try out the simple demos offered with the SDK, extract the release to any location on your computer, and run "TouchlessDemo.exe".

3. What's in the source repository?

The source repository is hosted at the homepage of Touchless:

<http://www.codeplex.com/touchless>

The directories you'll find there are:

- bin - Binary versions of the files, updated more frequently than our releases
- Documentation - Browse media documenting the project
- Samples - Apps/Games that demonstrate or use Touchless (submit yours!)
- TouchlessLib - Touchless functionality, written in C#, uses .NET 3.0+, VS2005+

- TouchlessUnitTests - Unit tests to prevent regression
- WebCamLib - Webcam functionality, written in C++, uses DX SDK Aug. 2007

Become a contributor by adding your Touchless apps or improving the SDK code!

4. How to add Touchless to your project

By adding Touchless to your project, you can give your users a truly fun, novel, and functional multi-touch experience, and all they need is a webcam!

The prerequisites you'll need to develop using Touchless are:

- Visual Studio 2005 or 2008
- .NET 3.0 or higher
- "TouchlessLib.dll" and "WebcamLib.dll"

To add Touchless to an existing visual Studio project, simply right click "References" and select "Add Reference..." go to the browse tab, and select "TouchlessLib.dll". Ensure that both "TouchlessLib.dll" and "WebcamLib.dll" are copied to the same output directory to be used with your builds.

5. Reference Information

You can find extensive information on all properties and methods of each class in our XML-generated documentation file "TouchlessLib.chm". The Touchless class diagram reflects public classes, methods, and properties of the API; it is available as "Touchless.jpg". These files are distributed with each release, and also available in the source code repository, located in the documentation folder.

6. Backlog of Ideas

This is a very terse list of ideas for improving Touchless SDK.

- ColorLib
 - Improve HSV colorspace partitioning model. We could group perceived similar colors better. Potentially replace with a group clustering algorithm. Perhaps just

refine the per-dimension bin counts, or replace the hash function.

- Use a lookup table instead of transforming RGB to HSV. We can just terminate early if it's not in the lookup table.
- Reduce loop overhead of converting ARGB values into RGB values, then into HSV values, then into Binned HSV values, then finally into a hash # for color lookup during marker update. Potentially use a lookup table for a subset of colors to avoid the math altogether.
- Improve HSV color grouping, consider refining the per-dimension bin counts or using a different HSV color-space partitioning model that better suits human perception of similar colors.

- **Marker**

- Implement a way of getting higher degree moments of inertia. Mostly, we are interested in the axis of least rotational momentum and the roundness factor.
- Allow the user to send a mask image with the add marker bitmap for arbitrary marker region selections."
- Extend or replace alpha smoothing with exponential decay to provide smoothed marker data and reduce the marker jumpiness.
- Optimize threshold, or replace threshold concept with a partial matching. Also, step threshold by numbers that actually make a difference, or just have sensitivity +/- buttons and increment functions
- Expose smoothing factor as a public marker property.
- Fix and improve the automated marker tests
- Standardize some marker colors, create an "auto-find makers"
- Improve the search bounds of a previously absent marker.
- Improve the meta-tracking (cases where small numbers of pixels are missing from the middle of a marker, or are outliers of the concentration of pixels)
- Periodically/continuously adopt surrounding pixels of confirmed marker pixels
- Coloravg is currently just marker representative color. Implement a way of actually getting a color average from the set of colors found
- Improve Marker highlighting
- Improve upon the raster scan algorithm used for marker updating.
- Optimize the method for getting the marker appearance from a circular area of a bitmap; we could use hierarchical bounds intersection or something smarter than the current scan algorithm.
- Optimize the values used to increment/decrement color frequencies for marker appearance detection. This should be somehow based on signal/noise ratios.
- Improve the expected marker regions used for scanning on update. We could consider the marker's acceleration, rather than just the velocity. Perhaps try using regions that aren't axis-aligned rectangles.

- **TouchlessMgr**

- Add functionality to save and load marker configuration files (reduce repeat

training of the same marker, possibly provide autoconfig files for standard markers... will variant lighting allow for this?)

- Implement additional marker data such as ColorAverage, ColorSpace, Axis and Roundness.
- Add flood fill algorithm so we can add a marker with a few points in the Bitmap.
- Refine the marker tracked colors as we find colors around the marker.
- The representative color doesn't always match the perceived color of the marker.
- Provide subsequent examples of a marker appearance
- Have TouchlessMgr actually expose a way to get a list of the current markers
- Make a better exception for camera start failure
- Validate the PixelFormat of incoming images.
- Create a utility function to retrieve ImageData in a consistent manner; we have a bit of code duplication right now.
- Make a public interface for demo classes to implement, then allow the user to just invoke start and stop of a demo class on the library
- Standardize error handling and exception generation across the project.

Camera

Class

Properties

- CaptureHeight
- CaptureWidth
- Fps

Methods

- Dispose
- GetCurrentImage
- ShowPropertiesDialog
- ToString

Events

- OnImageCaptured

TouchlessMgr

Class

Properties

- Cameras
- CurrentCamera
- MarkerCount
- Markers

Methods

- AddMarker
- Dispose
- RefreshCameraList
- RemoveMarker
- TouchlessMgr

Marker

Class

Properties

- CurrentData
- Highlight
- LastGoodData
- Name
- PreviousData
- ProvideCalculatedProperties
- RepresentativeColor
- SmoothingEnabled
- Threshold

Methods

- ToString

Events

- OnChange

MarkerEventArgs

Class

Event Args

Properties

- EventData
- EventMarker

MarkerEventData

Struct

Properties

- Area
- Bounds
- ColorAvg
- DX
- DY
- Present
- Timestamp
- X
- Y

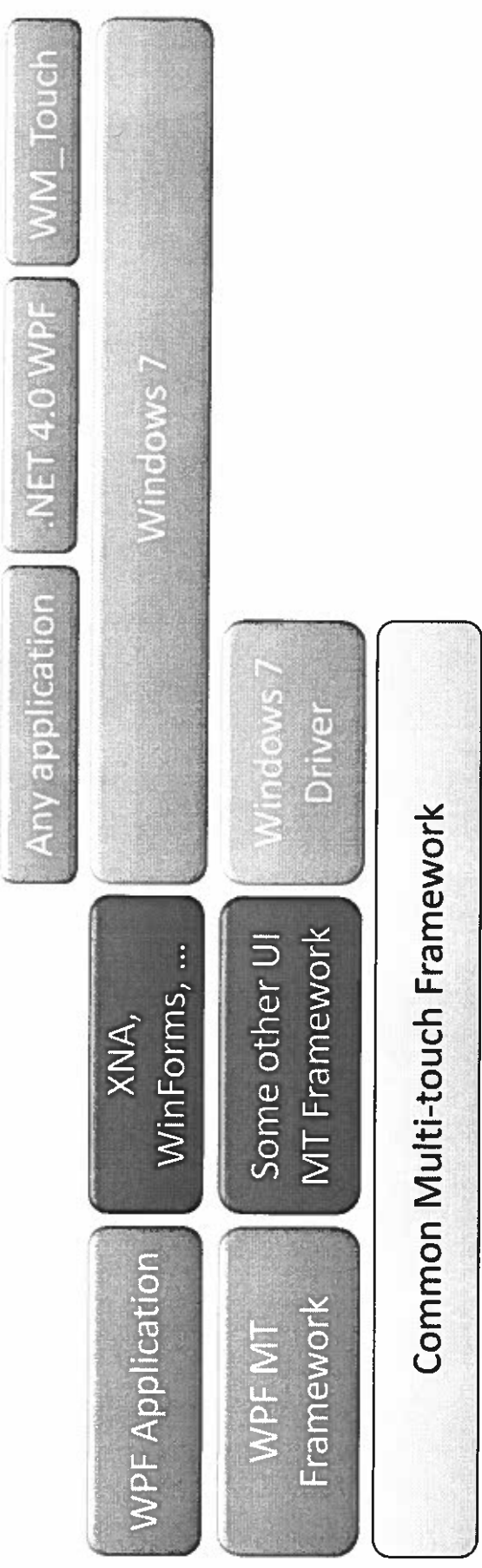
CameraEventArgs

Class

Event Args

Properties

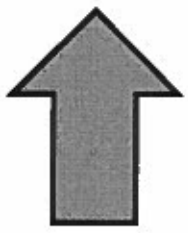
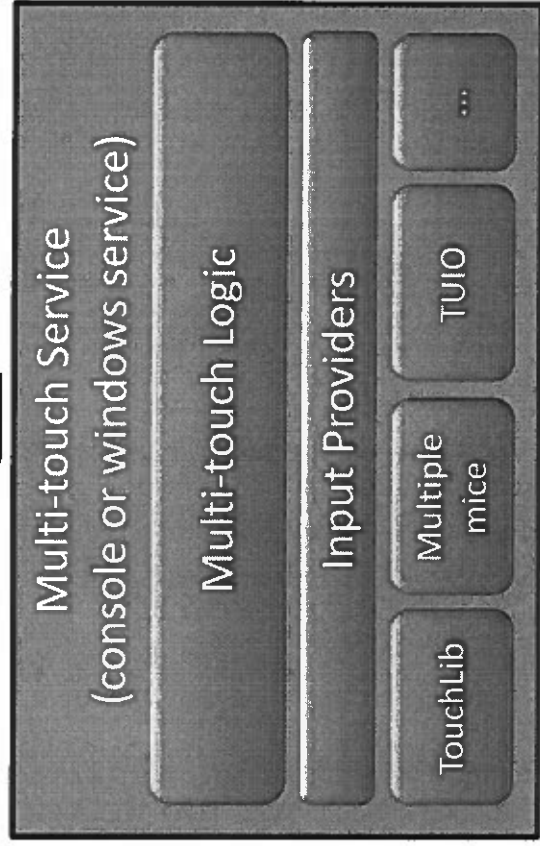
- Image



All interfaces are based on WCF.
Can transfer SOAP messages over
http, tcp, named pipe, ...



Application Interface

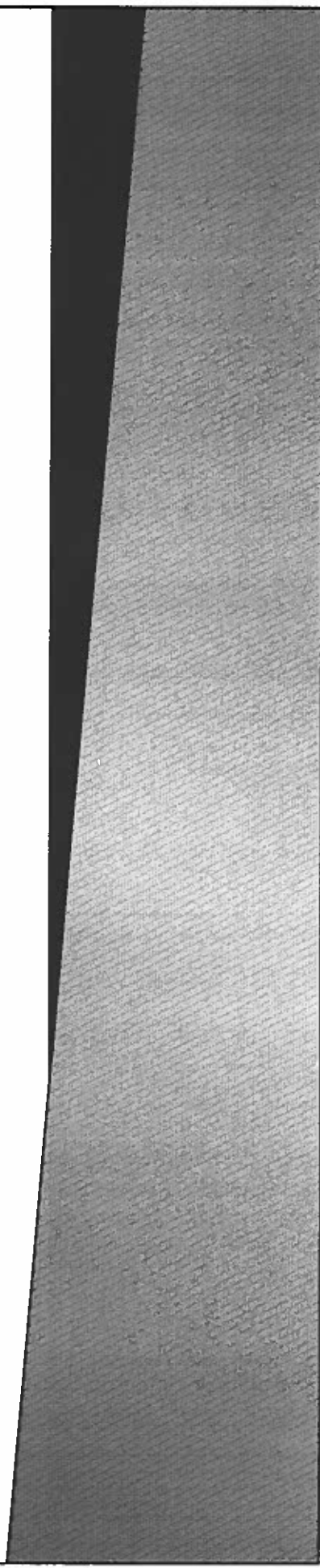


Configuration Application

Configuration Interface

Touchless SDK & Multi-Touch Vista

By:
Markowitz Frost
Advisor:
Dr. Yu-Ju Lin



Introduction

- ▶ Background
- ▶ Previous developer issues
- ▶ This is a project with much more to offer
- ▶ Integration with Windows 7 & 8

Project Summary

- ▶ Touchless SDK
 - Designed for use as multi-touch interface
 - Simple API
- ▶ Multi-Touch Vista
 - Input manager
 - Alternate interaction with Windows environment

Problems with Multi-Touch Vista

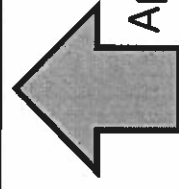
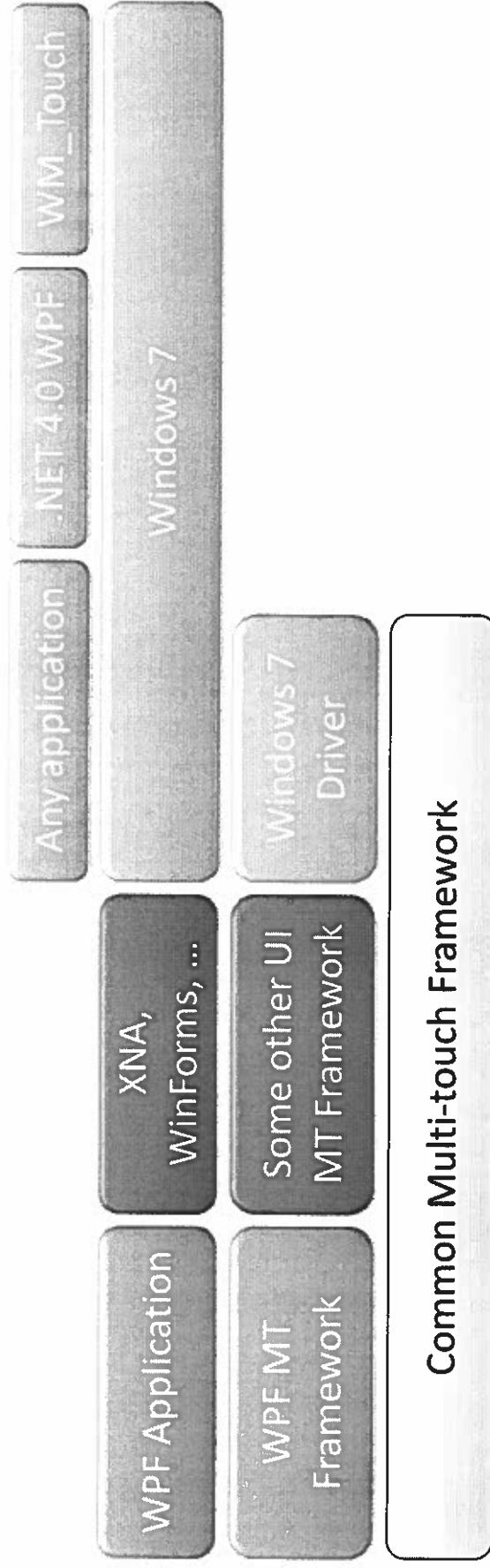
- ▶ Learning XAML to interface with WPF
 - Multi-Touch surface
- ▶ Custom input commands
 - Output standard commands/protocols
- ▶ Interfacing with Touchless SDK
 - Requires a lot of effort

Similar Technology

- ▶ Microsoft Kinect
 - May seem similar but not quite
- ▶ Sony Playstation Move
 - Closest relation to Touchless
- ▶ Android 4.0 Face unlock
 - Uses object recognition

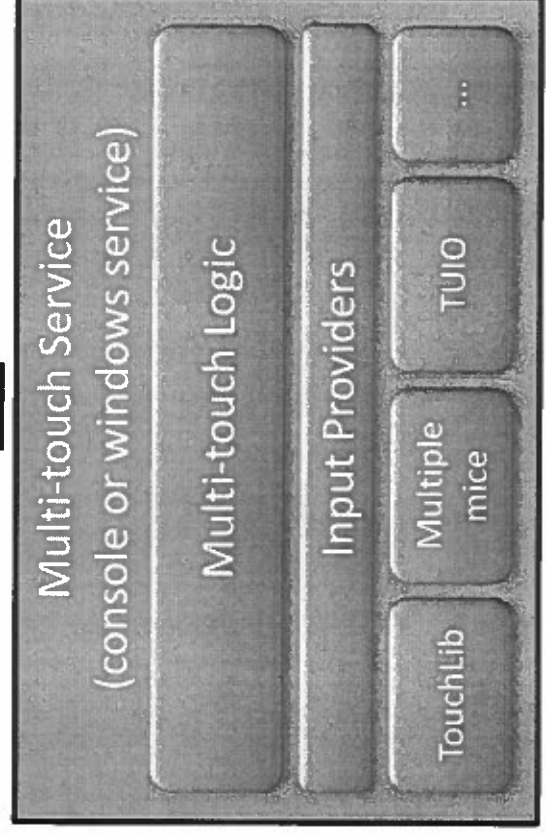
Problems with Touchless SDK

- ▶ Inaccurate raw data
 - Cursor not entirely useful
- ▶ Relative mouse algorithm
 - Was not useable
- ▶ Mouse movement was not smooth
 - Jumped around when receiving coordinates
- ▶ Second marker burden
 - Used too many CPU cycles



Application Interface

All interfaces are based on WCF.
Can transfer SOAP messages over
http, tcp, named pipe, ...



Configuration Application

Configuration Interface

Using Touchless SDK

- ▶ Simple interface to jump into and go
 - Small sacrifice for accuracy
- ▶ Responsive, not terribly CPU intensive
- ▶ Very flexible operation
- ▶ If light is good enough, track anything....

ANYTHING....

- ▶ Enjoy the sample!

While Multi-Touch Vista did not work...

- ▶ Wanted to show that Touchless is good for real world interaction
- ▶ Proof of concept for Dr. Lin and Brandon's senior project
- ▶ Enjoy the show!

Q & A

